



Advance Software Engineering Microservices

By:
Dr. Salwa Osama



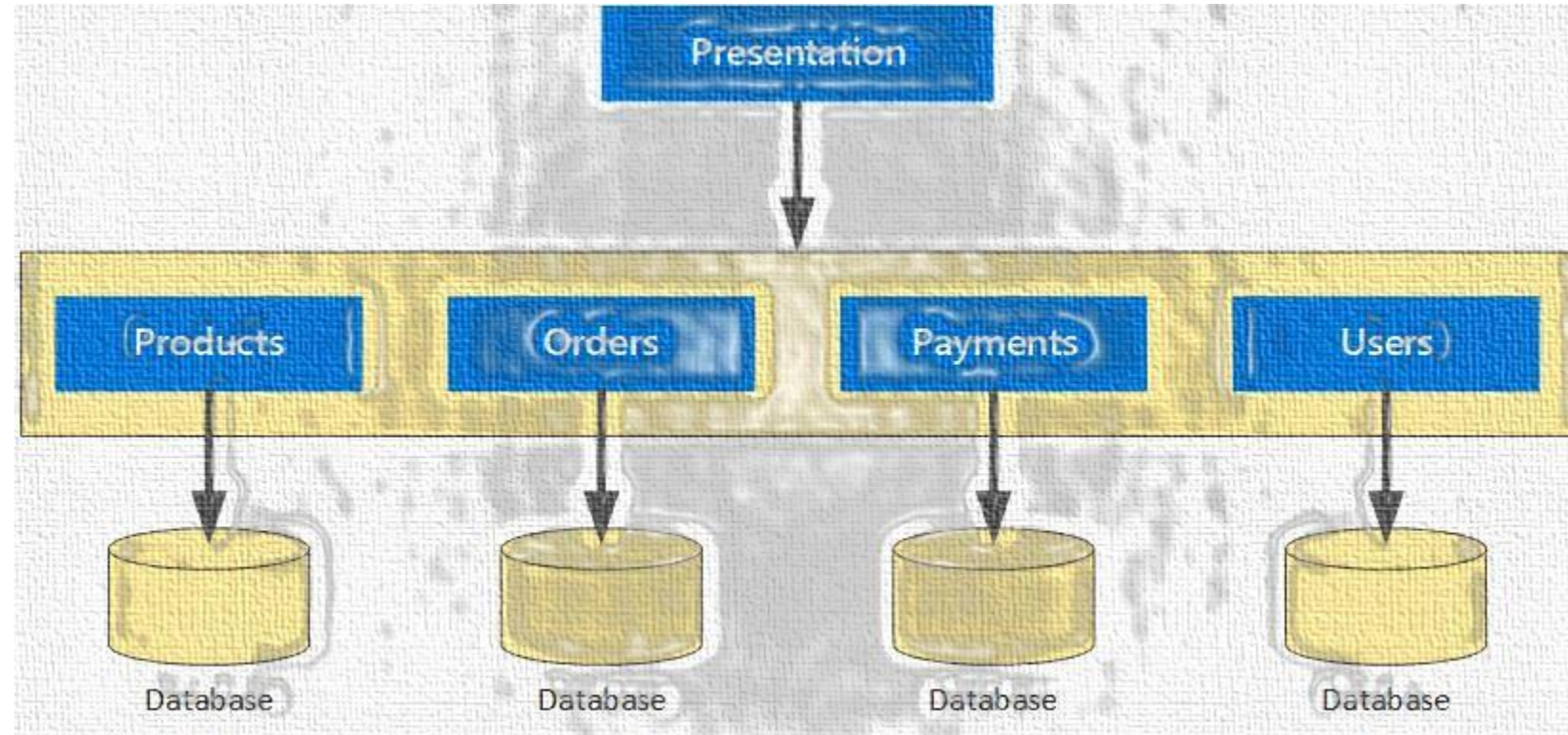
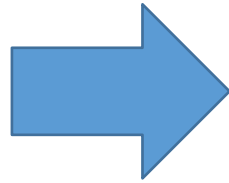
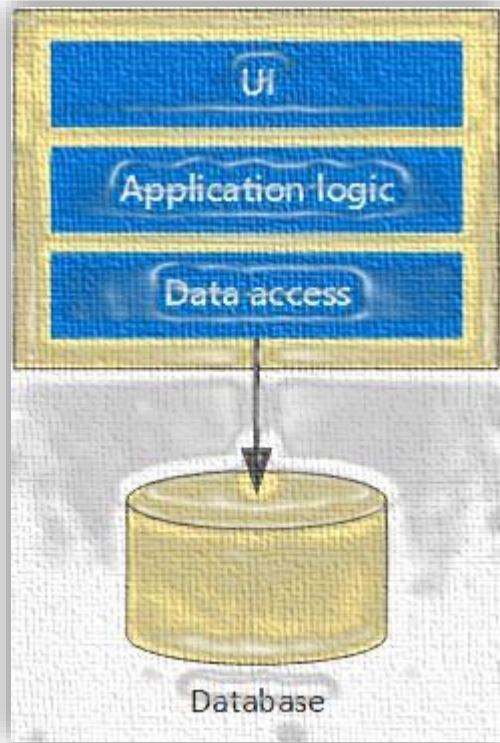
Monolithic

- **Monolithic application** describes a single-tiered software application in which the user interface and data access code are combined into a single program from a single platform.
- A monolithic application is self-contained and independent from other computing applications. The design philosophy is that the application is responsible not just for a particular task, but can perform every step needed to complete a particular function

Microservices

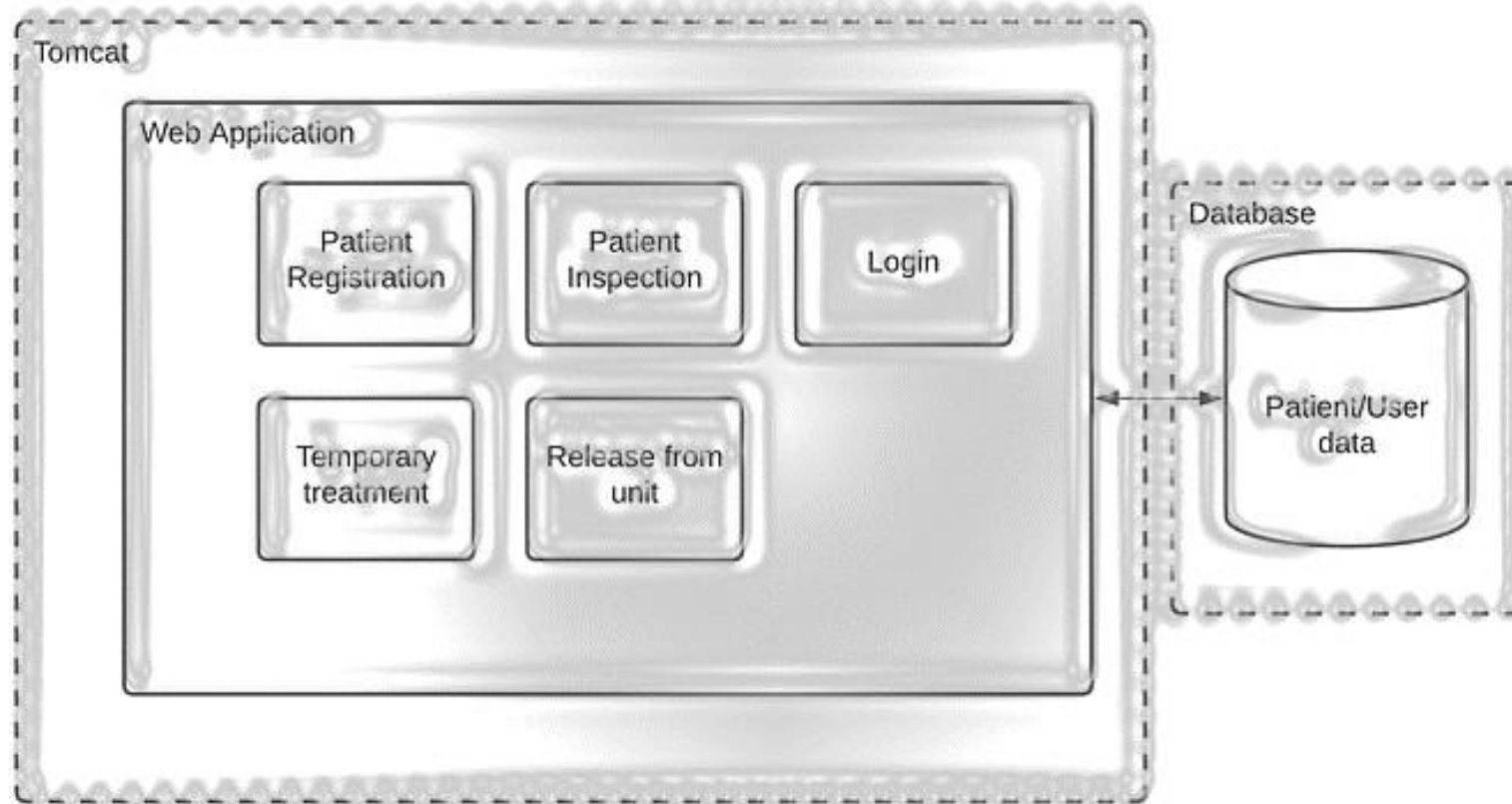
- Microservices are small, independent, and loosely coupled.
 - A single small team of developers can write and maintain a service.
- Each service is a separate codebase,
 - which can be managed by a small development team.
- Services can be deployed independently.
 - A team can update an existing service without rebuilding and redeploying the entire application.
- Services are responsible for persisting their own data or external state.
 - This differs from the traditional model, where a separate data layer handles data persistence.
- Services communicate with each other by using well-defined APIs.
 - Internal implementation details of each service are hidden from other services.
- Supports polyglot programming.
 - For example, services don't need to share the same technology stack, libraries, or frameworks.

From monolithic to Microservices



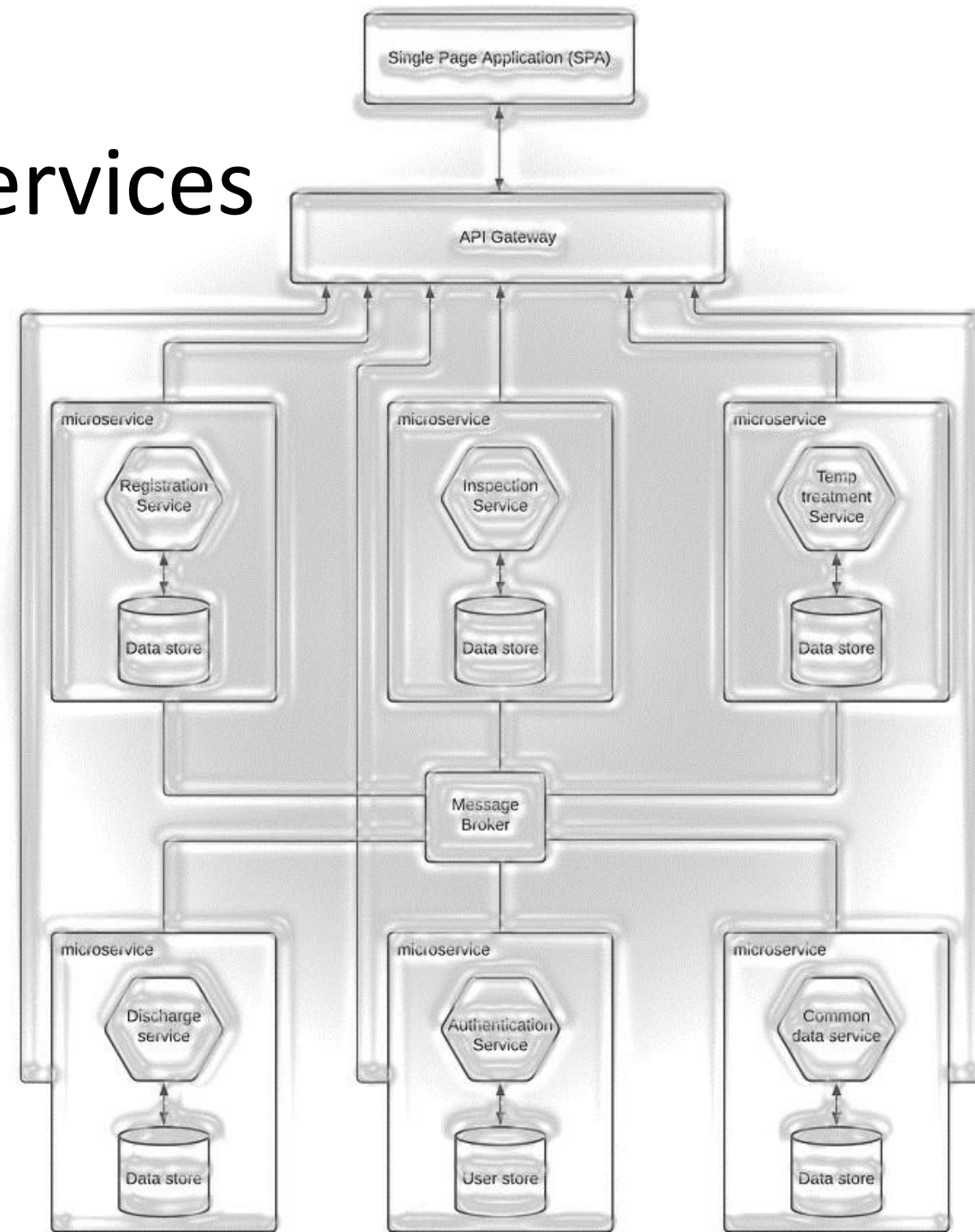
From monolithic to Microservices

- Example: Hospital OPD (OutPatient Department)



From monolithic to Microservices

- Example: Hospital OPD (OutPatient Department)



From monolithic to Microservices

Functional breakdown of authentication features

User registration

Set up new login ID

Set up new password

Set up password recovery information

Set up two-factor authentication

Confirm registration

Authenticate using UID/password

Get login ID

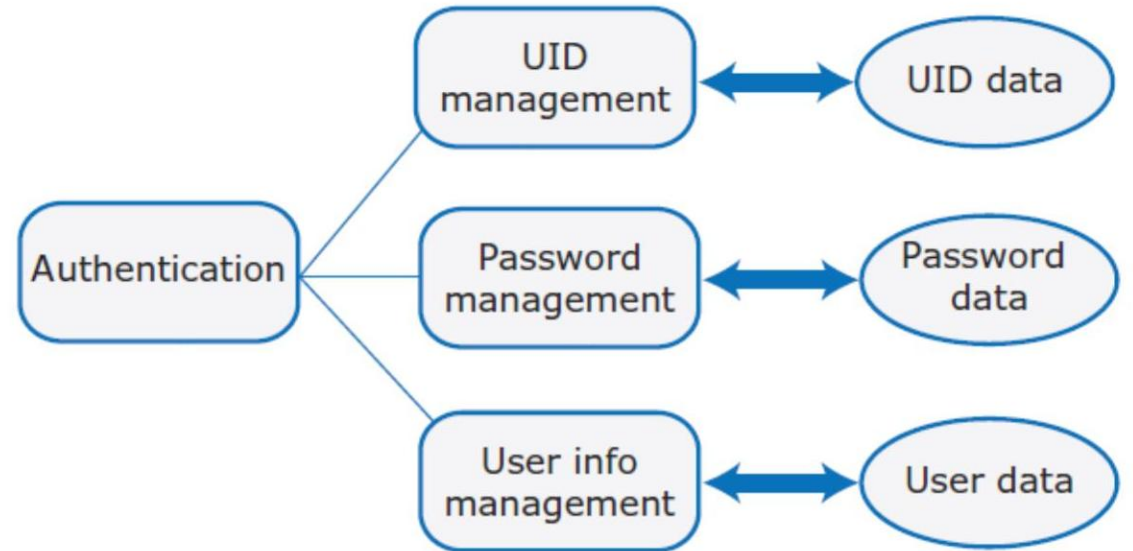
Get password

Check credentials

Confirm authentication



Authentication microservices



Benefits of Microservices Architecture



Faster Deployments

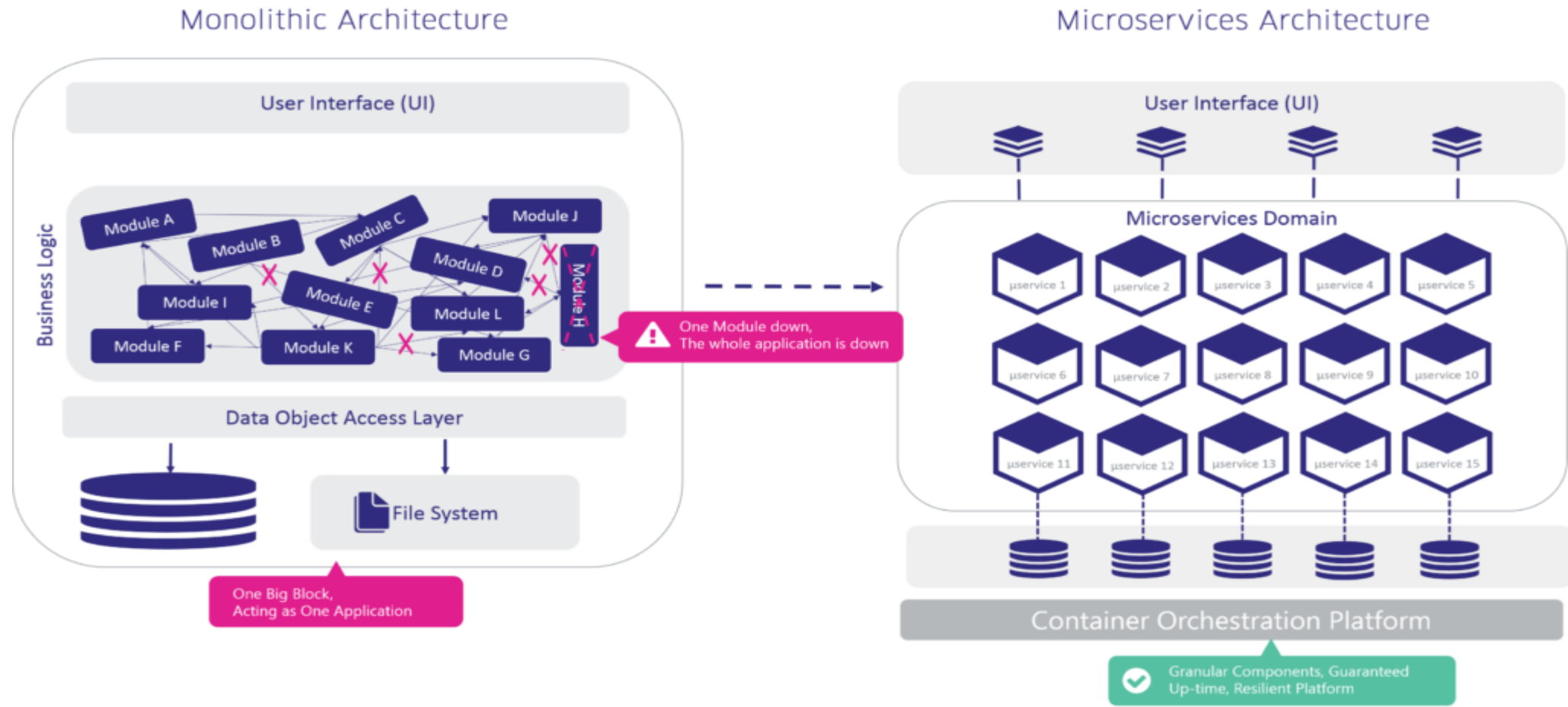
- Microservices architecture allows for faster development and deployments since it simplifies the design.
- Each module is easily built, tested and deployed independently, thus giving your business increased agility and faster time to market.

Scalability

- Microservices architecture by design offers ease in scalability since the modules work independently and each has bounded context.
- The design is far more simple allowing fast adding of new features.
- With test automation each module is tested independently and the whole system is tested in minutes automatically thus cutting down the time to build, test and release new features.

Guaranteed up-time

- With the right microservices architecture, it's almost impossible for the whole system to go down. Since each microservice is built, deployed and managed independently and is totally isolated at runtime so the whole system can never go down.



No Vendor Lock

- Microservices architecture and principles are based on open standards and backed up heavily by the open source community.
- Almost all commercial microservices platforms follow these open standards which makes it easy to migrate from them if needed.
- Also by design microservices have standard based APIs which makes it easy to extend and even use different programming languages and technologies.

Resilience & Robustness

- With microservices architecture, your platforms are far more resilient and robust.
 - In other words, Kubernetes or OpenShift Container platforms are monitoring and self-healing whenever the system is affected.
- Their orchestration capabilities can self-monitor and restart failed components. Self-healing extends from monitoring infrastructure, clusters to nodes and pods.
- Self-monitoring and healing ensures resilience for your applications and systems.

Dynamic Talent Pool

- With microservices, it's easier to onboard new team members as well as hiring software engineers with different coding backgrounds.

Challenges in Microservices

- **Complexity.** A microservices application has more moving parts than the equivalent monolithic application. Each service is simpler, but the entire system as a whole is more complex.
- **Development and testing.** Writing a small service that relies on other dependent services requires a different approach than a writing a traditional monolithic or layered application. Existing tools are not always designed to work with service dependencies. Refactoring across service boundaries can be difficult. It is also challenging to test service dependencies, especially when the application is evolving quickly.

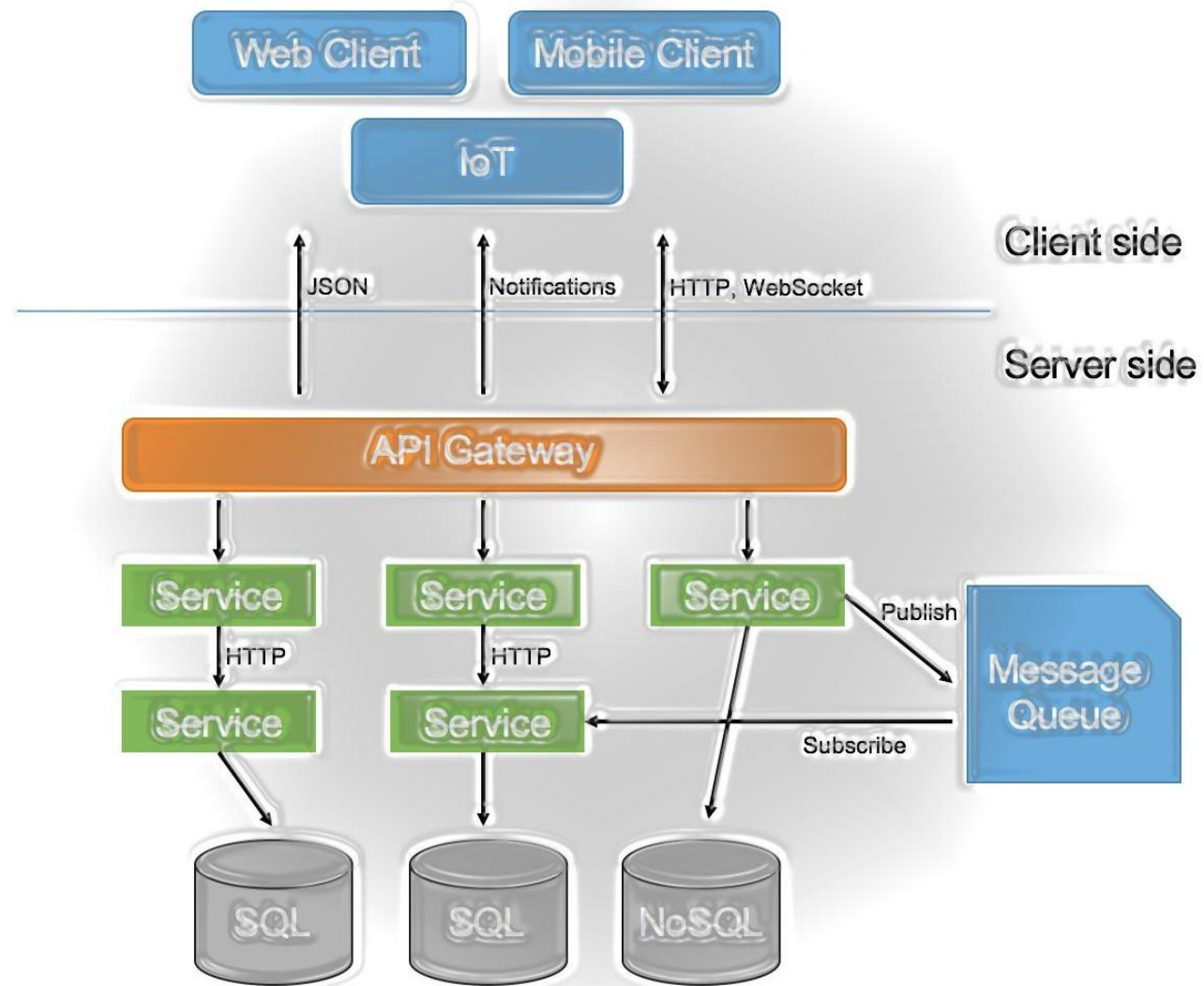
Challenges

- **Network congestion and latency.** The use of many small, granular services can result in more interservice communication. Also, if the chain of service dependencies gets too long (service A calls B, which calls C...), the additional latency can become a problem. You will need to design APIs carefully. Avoid overly chatty APIs, think about serialization formats, and look for places to use asynchronous communication patterns like queue-based load leveling.
- **Data integrity.** With each microservice responsible for its own data persistence. As a result, data consistency can be a challenge.

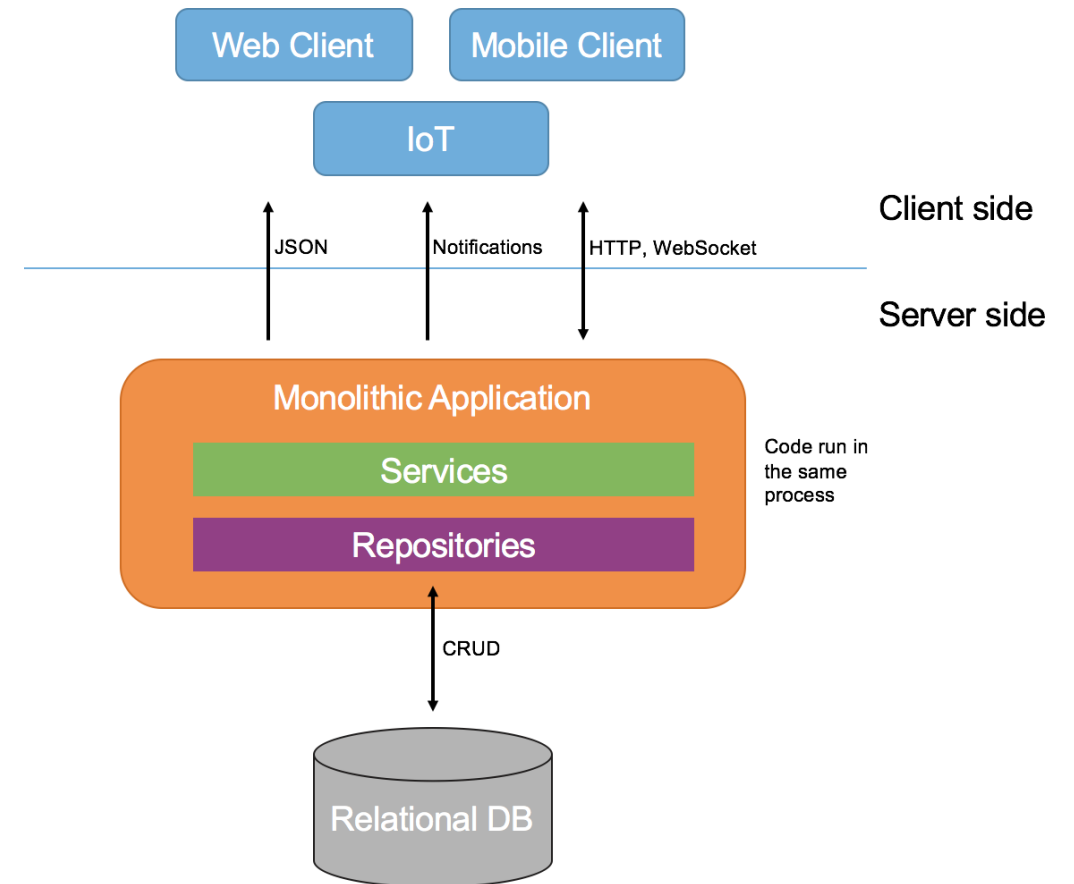
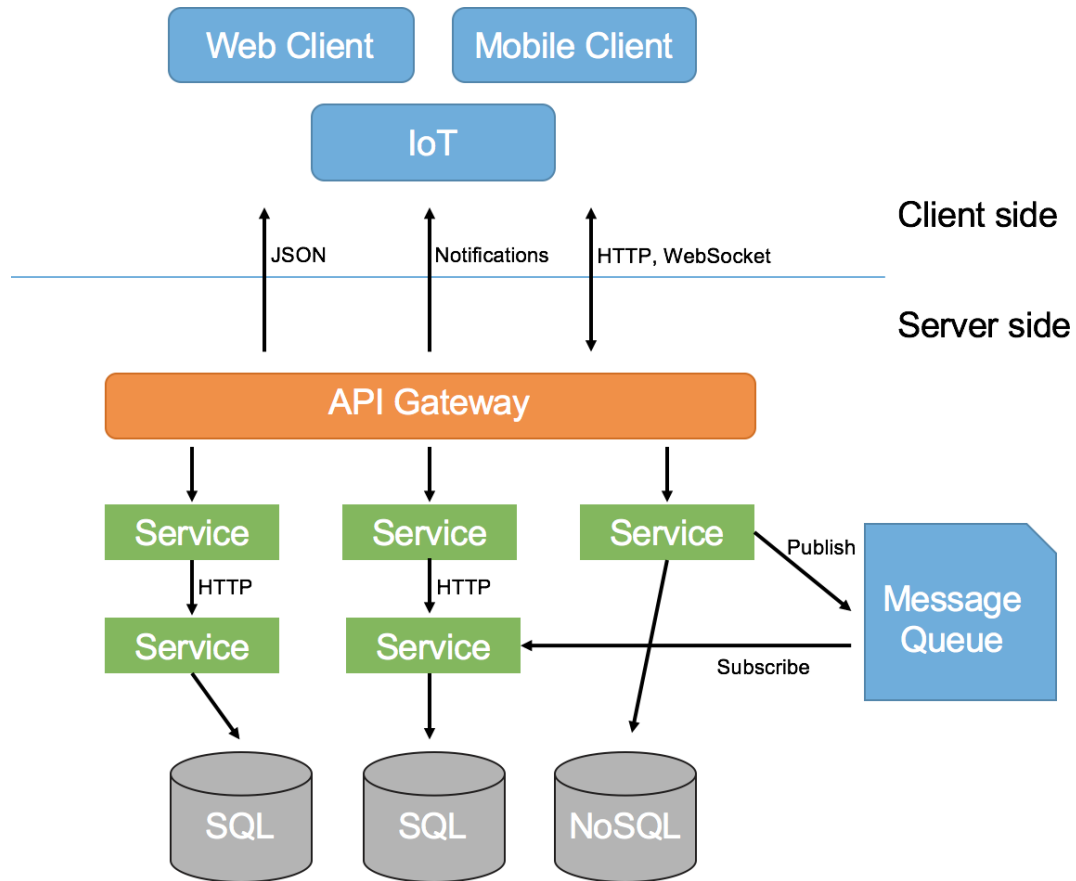
Challenges

- **Versioning.** Updates to a service must not break services that depend on it. Multiple services could be updated at any given time, so without careful design, you might have problems with backward or forward compatibility.
- **Skill set.** Microservices are highly distributed systems. Carefully evaluate whether the team has the skills and experience to be successful
- **Lack of governance.** The decentralized approach to building microservices has advantages, but it can also lead to problems. You might end up with so many different languages and frameworks that the application becomes hard to maintain.

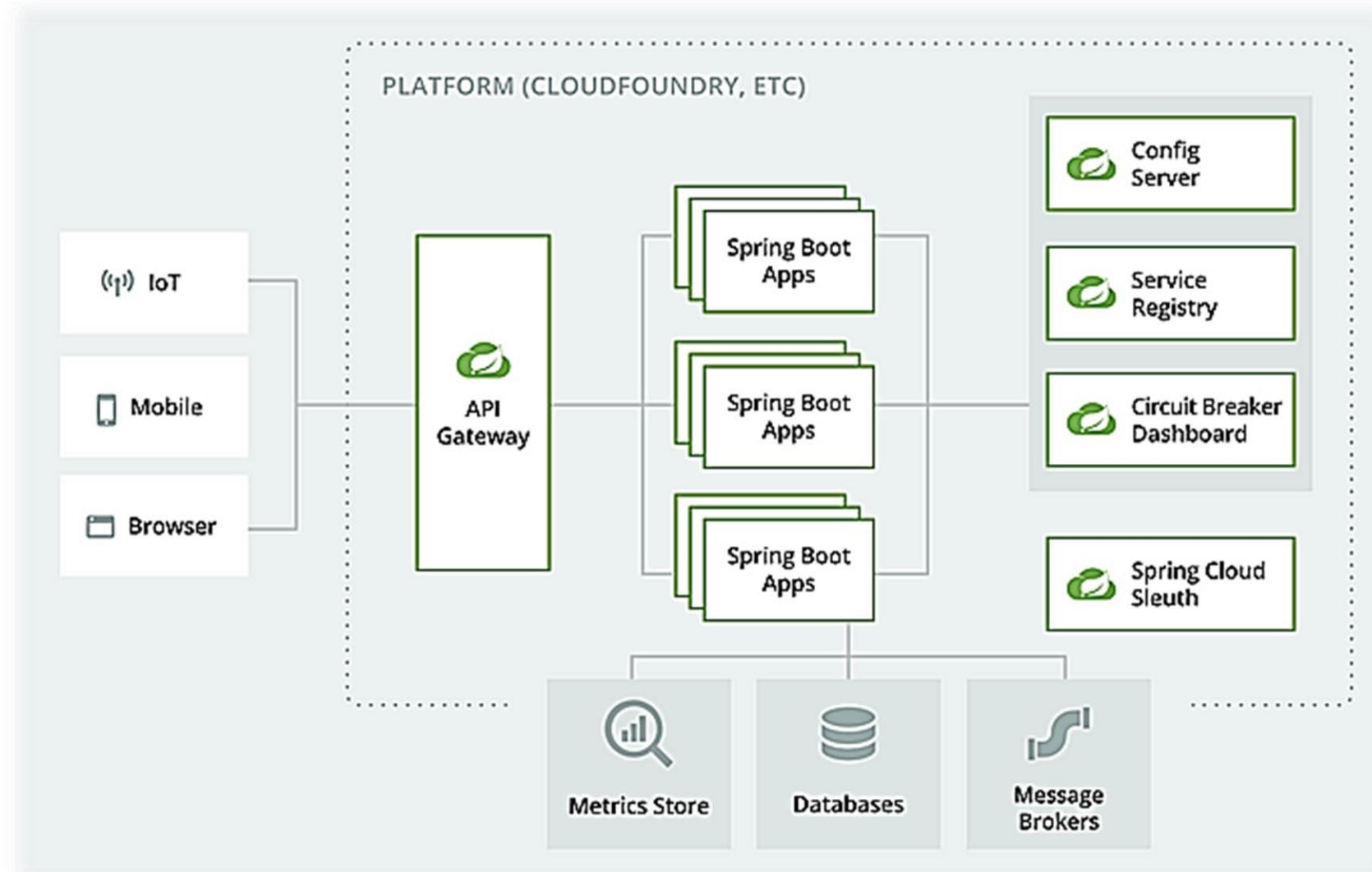
Microservice Architecture



Microservice VS Monolithic Architecture



Microservice resilience with Spring Cloud



Microservice Characteristics

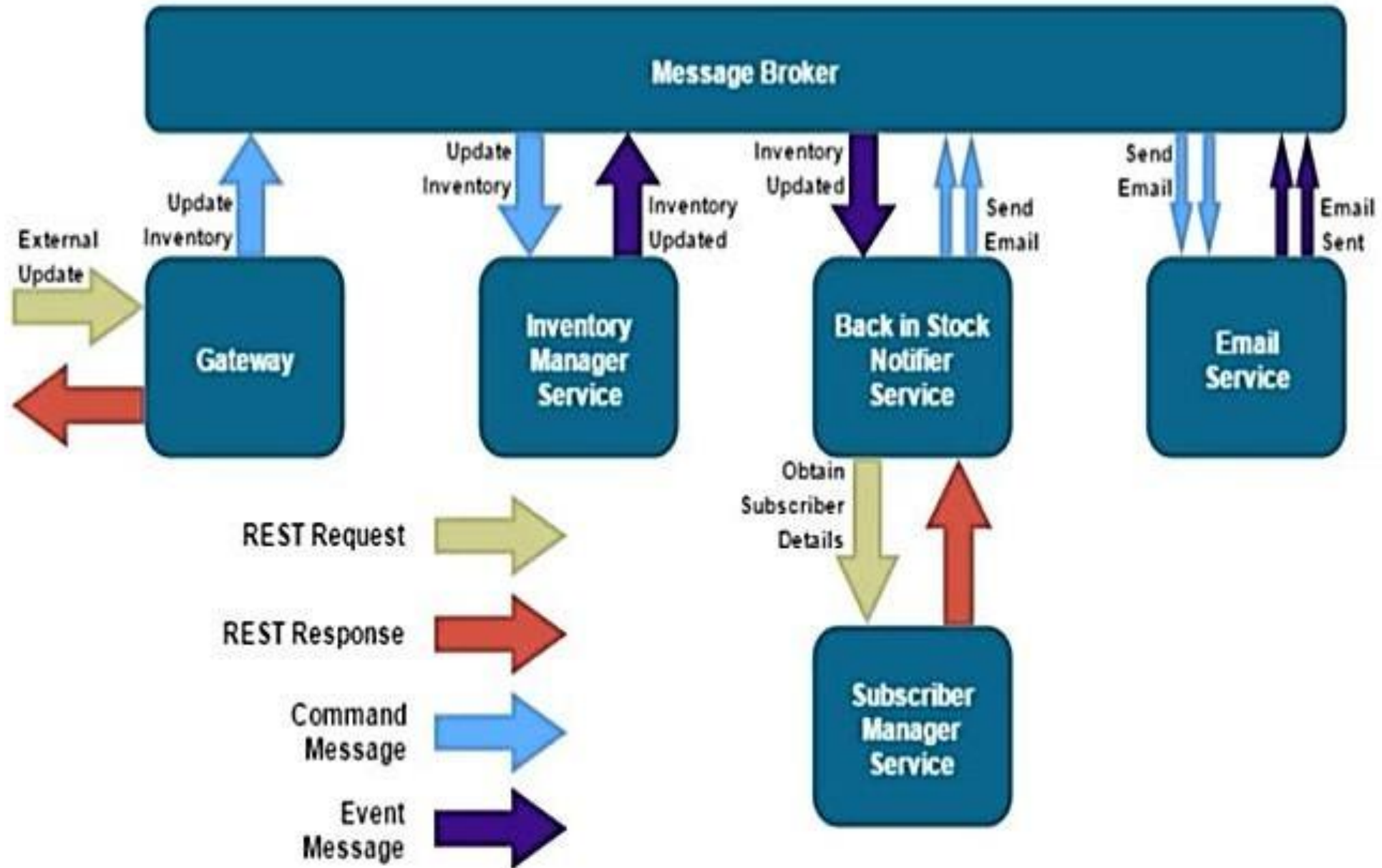
- Single responsibility
 - Business Boundary (for logic Business- DDD)
 - Function Boundary (for development)
- Domain-driven design (DDD) is a major software design approach, focusing on modeling software to match a domain according to input from that domain's experts.
 - Under domain-driven design, the structure and language of software code (class names, class methods, class variables) should match the business domain.

Communication Design

- Synchronous communication. In this pattern, a service calls an API that another service exposes, using a protocol such as HTTP or gRPC.
 - This option is a synchronous messaging pattern because the caller waits for a response from the receiver.
- Asynchronous message passing. In this pattern, a service sends message without waiting for a response, and one or more services process the message asynchronously.

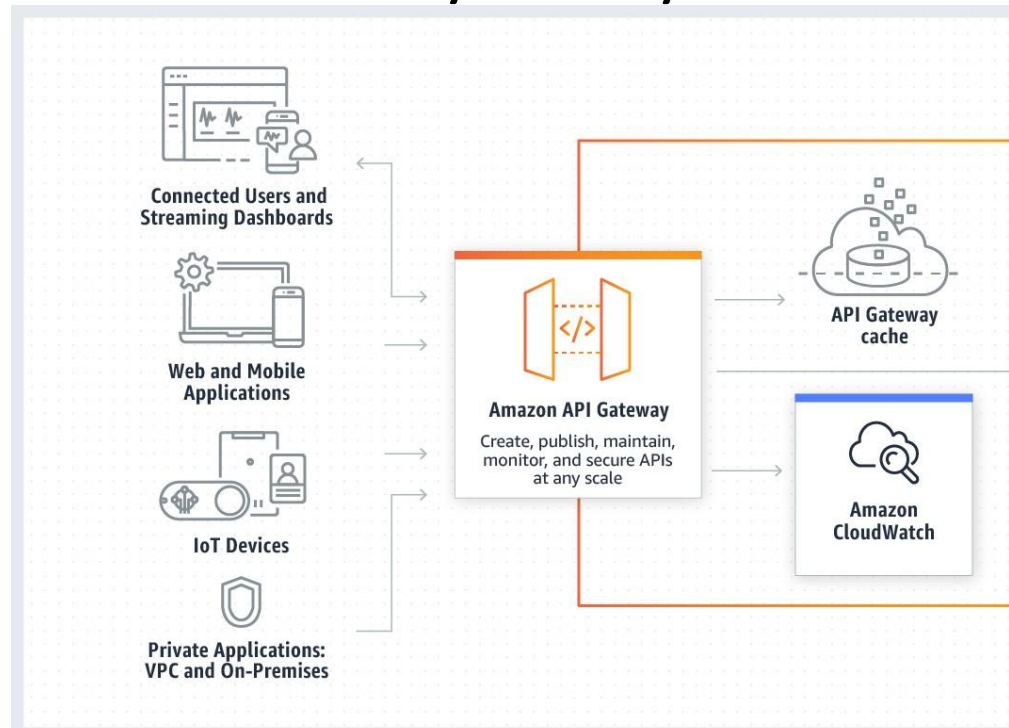
Communication design

- HTTP Communication:
 - Also known as Synchronous communication, the calls between services is a viable option for service-to-service via REST API.
- Message communication:
 - Also known as Asynchronous communication, the services push messages to a message broker that other services subscribe to.
- Event-driven communication:
 - Another type of Asynchronous communication, the services don't need to know the common message structure



API Gateway

- API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. APIs act as the "front door" for applications to access data, business logic, or functionality from your backend services.



API Gateway products

“Microsoft
azure”
The Best one
supports
Dashboard
configuration

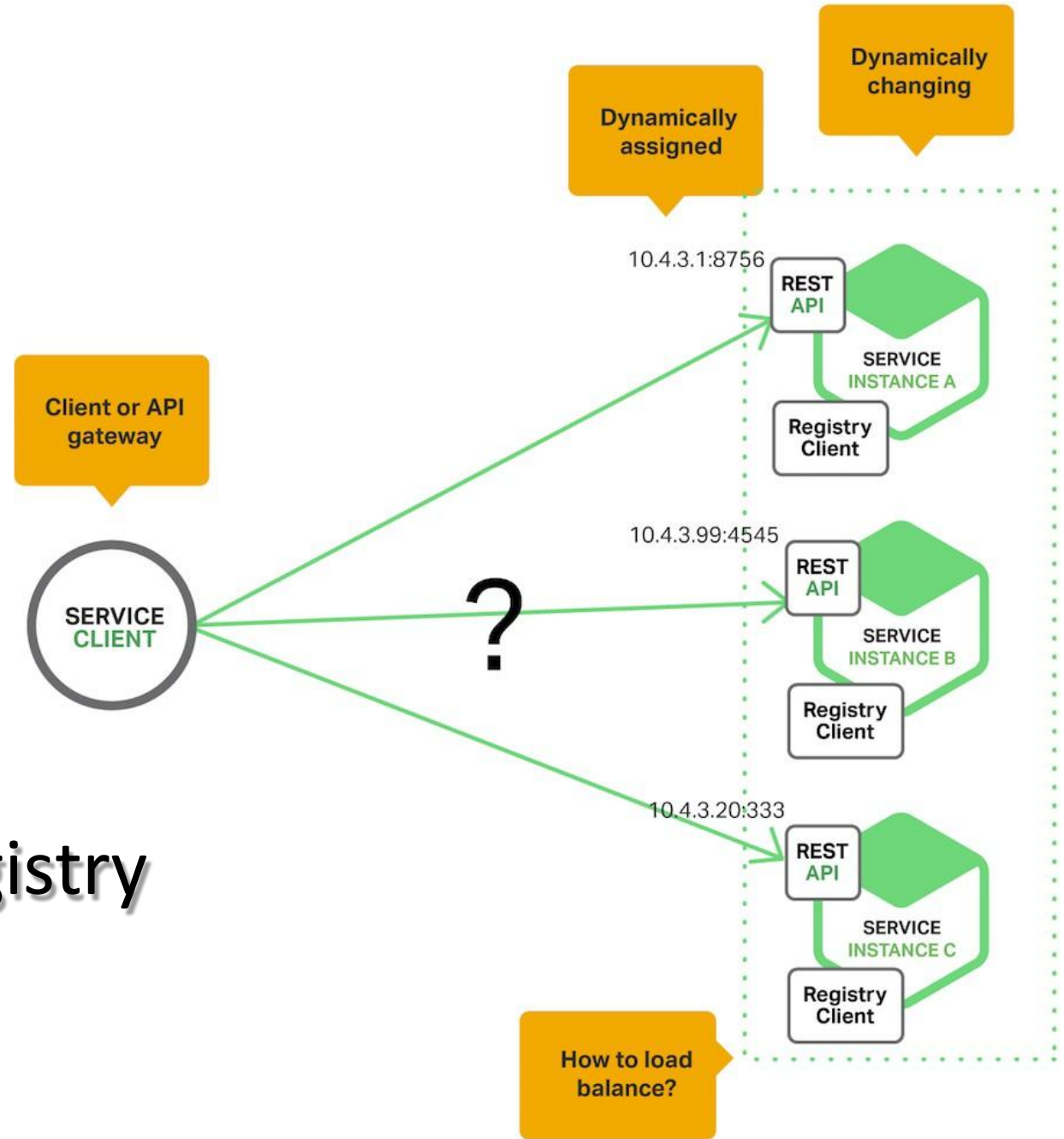


Service Discovery

- Let's imagine that you are writing some code that invokes a service that has a REST API. In order to make a request, your code needs to know the network location (IP address and port) of a service instance.
- In a traditional application running on physical hardware, the network locations of service instances are relatively static. For example, your code can read the network locations from a configuration file that is occasionally updated.
- In a modern, cloud-based microservices application, however, this is a much more difficult problem to solve.
 - Service instances have dynamically assigned network locations. Moreover, the set of service instances changes dynamically because of autoscaling, failures, and upgrades. Consequently, your client code needs to use a more elaborate service discovery mechanism.

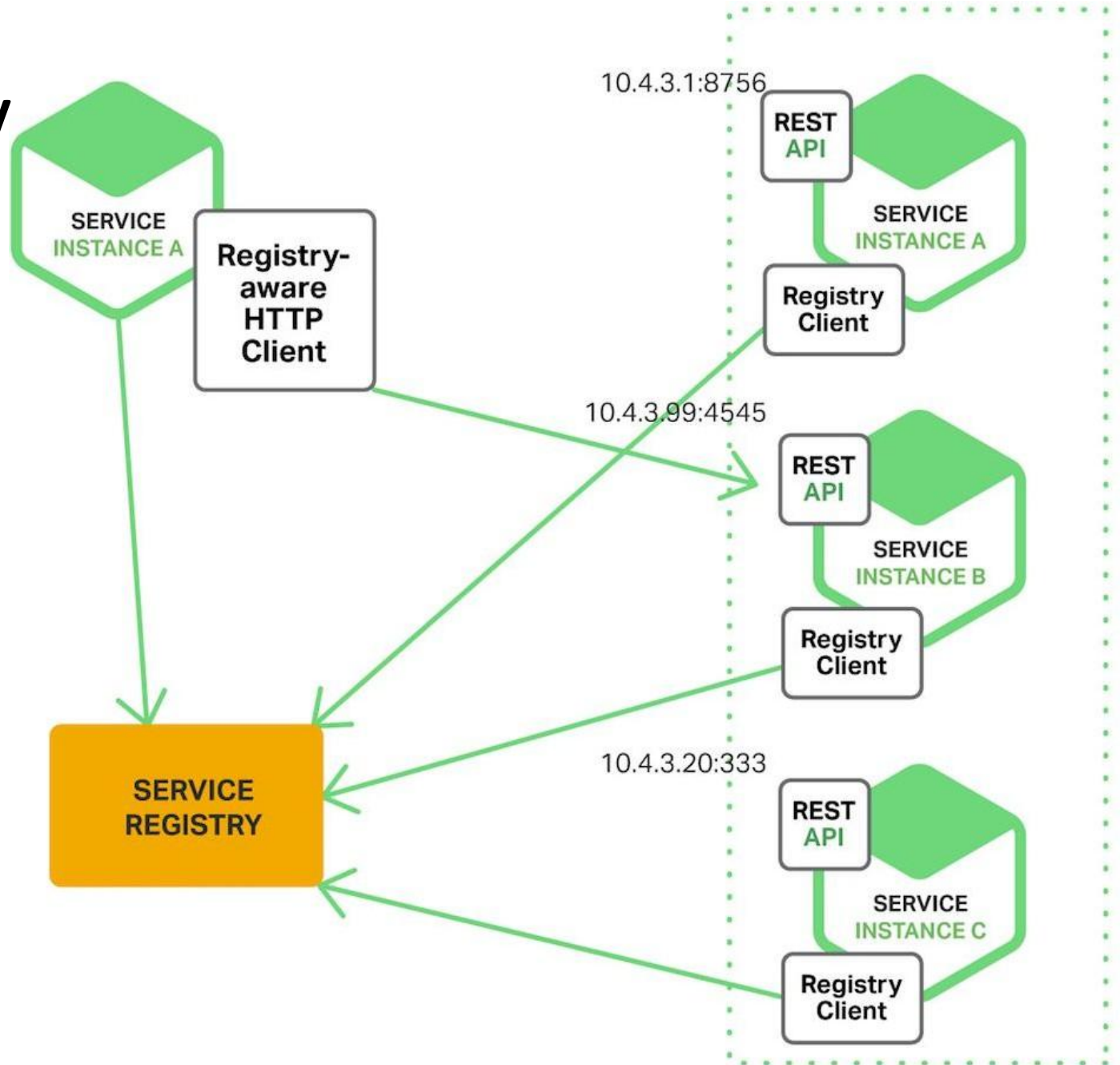
Service Discovery

Solution Using Service registry



Service Discovery

- Load Balance
- Resilient
- Fault-tolerant



Service Discovery

) Load balanced

- dynamically load balance requests across all service instances to ensure that the service invocations are spread across all the service instances managed by it.

) Resilient

- client should “cache” service information locally. Local caching allows for gradual degradation of the service discovery feature so that if service discovery service does become unavailable, applications can still function and locate the services based on the information maintained in its local cache.

) Fault-tolerant

- detect when a service instance isn't healthy and remove the instance from the list of available services.

